MAR BASELIOS

INSTITUTE OF TECHNOLOGY AND SCIENCE

NELLIMATTOM P.O., KOTHAMANGALAM

PHONE : 0485 – 2859348, 2859318

LABORATORY RECORD

YEAR.....

Name:..... Roll No:.....

Branch: ..... Batch: .....

MAR BASELIOS

INSTITUTE OF TECHNOLOGY AND SCIENCE

NELLIMATTOM P.O., KOTHAMANGALAM

PHONE : 0485 – 2859348, 2859318

LABORATORY RECORD

YEAR.....

Name:..... Roll No:.....

Branch: ..... Batch: .....

Certified that this is the Bonafide Record of Practical work done in the …….. Laboratory Mar Baselios Institute of Technology and Science Nellimattom P.O, Kothamangalam, by ……..

Head of the Department **Staff Member in Charge**

Uni. Ex. Reg:No……ofMarch/Sept……...

Uni. Ex. Reg:No……of March/Sept…….....

Internal Examiner External Examiner

INDEX

S.No.	Date	Title	Page No.	Teacher's Sign/Remarks
1	22-01-2026	Execute DDL statements	1	
2	29-01-2026	Execute DML statements(Select,Insert,Update,Delete)	4	
3	29-01-2026	Create a table and execute DCL statements (grant,revoke)	7	
4	29-01-2026	Create a table and execute TCL statements	10	
5	19-02-2026	Accessing database (SELECT, Filtering using WHERE, HAVING, GROUP BY,ORDER BY Clauses)	13	
6	21-02-2026	Create a view of the tables given	17	
7	21-02-2026	Subquery in MySQL	25	
8	21-02-2026	Use join queries in MySQL	28	
9	21-02-2026	PL/SQL procedure to display all fields from a table	34	
10	21-02-2026	PL/SQL procedure to add two numbers	36	
11	21-02-2026	PL/SQL procedure to check whether a number is odd or even	37	
12	21-02-2026	PL/SQL procedure to find Factorial of a number	38	
13	21-02-2026	PL/SQL procedure to find maximum of three values	39	
14	21-02-2026	PL/SQL procedure to find the sum of digits	40	
15	04-03-2026	SQL Trigger : Execution of trigger	41	
16	04-03-2026	MySQL Trigger : BEFORE UPDATE	44	
17	09-03-2026	AFTER UPDATE Trigger	47	

18	09-03-2026	Program in PL/SQL to list the name of students using cursor	50	
19	09-03-2026	Program in PL/SQL to find average salary using cursor	52	
20	16-03-2026	Create a database and collection in MongoDB	54	
21	16-03-2026	Designing NoSQL Database - Employee Management	56	
22	18-03-2026	Performing CRUD Operations - Product Catalog	59	
23	23-03-2026	Usage of aggregate functions	62	
24	01-04-2026	Create users and roles in the employee database	66	
25	06-04-2026	Perform Replication, Indexing, Sharding and Clustering in MongoDB using shell commands	69	
26	08-04-2026	Develop a simple PHP application using MongoDB (NoSQL) to insert and retrieve student records	74	

Experiment - 1

Aim :

Execute DDL Statements.

Program Code :

i) Create a table student with fields (Roll No, Name, Age, course, year)

mysql> create database cenna;

Query OK, 1 row affected (0.03 sec)

mysql> use cenna;

Database changed

mysql> create table student(RollNo int primary key, Name varchar(100), Age int, course varchar(100), year int);

Query OK, 0 rows affected (0.11 sec)

mysql> desc student;

```
+--------+--------------+------+-----+---------+-------+
| Field  | Type         | Null | Key | Default | Extra |
+--------+--------------+------+-----+---------+-------+
| RollNo | int          | NO   | PRI | NULL    |       |
| Name   | varchar(100) | YES  |     | NULL    |       |
| Age    | int          | YES  |     | NULL    |       |
| course | varchar(100) | YES  |     | NULL    |       |
| year   | int          | YES  |     | NULL    |       |
+--------+--------------+------+-----+---------+-------+
```

5 rows in set (0.01 sec)

ii) Alter table

mysql> alter table student add marks int;

Query OK, 0 rows affected (0.22 sec)

Records: 0 Duplicates: 0 Warnings: 0

mysql> desc student;

Field	Type	Null	Key	Default	Extra
RollNo	int	NO	PRI	NULL	
Name	varchar(100)	YES		NULL	
Age	int	YES		NULL	
course	varchar(100)	YES		NULL	
year	int	YES		NULL	
marks	int	YES		NULL	

6 rows in set (0.01 sec)

mysql> insert into student (RollNo, Name, Age, course, year, marks)

values

(101,'Aparna',21,'MCA',2,80),

(102,'gopika',21,'MCA',2,81),

(103,'vishnupriya',21,'MCA',2,80),

(104,'Ashli',22,'MCA',2,81),

(105,'Aiswarya',22,'MCA',2,80);

Query OK, 5 rows affected (0.02 sec)

Records: 5 Duplicates: 0 Warnings: 0

mysql> select * from student;

RollNo	Name	Age	course	year	marks
101	Aparna	21	MCA	2	80
102	gopika	21	MCA	2	81
103	vishnupriya	21	MCA	2	80
104	Ashli	22	MCA	2	81
105	Aiswarya	22	MCA	2	80

5 rows in set (0.00 sec)

iii) Truncate table

mysql> truncate table student;

Query OK, 0 rows affected (0.16 sec)

mysql> desc student;

Field	Type	Null	Key	Default	Extra
RollNo	int	NO	PRI	NULL	
Name	varchar(100)	YES		NULL	
Age	int	YES		NULL	
course	varchar(100)	YES		NULL	
year	int	YES		NULL	
marks	int	YES		NULL	

5 rows in set (0.01 sec)

iv) Drop table

mysql> drop table student;

Query OK, 0 rows affected (0.06 sec)

mysql> desc student;

ERROR 1146 (42S02): Table cenna.student' doesn't exist

Experiment - 2

Aim :

Execute DML Statements (Select , Insert , Update , Delete)

Program Code :

i) Create table Employee (Emp_Id, Emp_Name, Dept_Id, Salary) and also create another table Department (Dept_Id, Dept_Name, Dept_Head)

mysql> create table Department(Dept_Id int primary key, Dept_Name varchar(50), Dept_Head varchar(50));
Query OK, 0 rows affected (0.11 sec)

mysql> create table Employee(Emp_Id int primary key, Emp_Name varchar(50), Dept_Id int, Salary float, foreign key(Dept_Id) references Department(Dept_Id));
Query OK, 0 rows affected (0.12 sec)

mysql> desc Department;

Field	Type	Null	Key	Default	Extra
Dept_Id	int	NO	PRI	NULL	
Dept_Name	varchar(50)	YES		NULL	
Dept_Head	varchar(50)	YES		NULL	

3 rows in set (0.00 sec)

mysql> desc Employee;

Field	Type	Null	Key	Default	Extra
Emp_Id	int	NO	PRI	NULL	
Emp_Name	varchar(50)	YES		NULL	
Dept_Id	int	YES	MUL	NULL	
Salary	float	YES		NULL	

4 rows in set (0.00 sec)

ii) Insert minimum of 4 rows

mysql> insert into Department(Dept_Id,Dept_Name,Dept_Head) values (1,'Dept_A','Dept_HeadA'),(2,'Dept_B','Dept_HeadB'),(3,'Dept_C','Dept_HeadC'),(4,'Dept_D','Dept_HeadD');

Query OK, 4 rows affected (0.02 sec)

Records: 4 Duplicates: 0 Warnings: 0

mysql> insert into Employee(Emp_Id,Emp_Name,Dept_Id,Salary) values (101,'EmpA',1,50000.00),(102,'EmpB',2,35000.00),(103,'EmpC',3,70000.00),(104,'EmpD',4,50000.00);

Query OK, 4 rows affected (0.02 sec)

Records: 4 Duplicates: 0 Warnings: 0

iii) Display the records

mysql> select * from Department;

```
+---------+-----------+------------+
| Dept_Id | Dept_Name | Dept_Head  |
+---------+-----------+------------+
|       1 | Dept_A    | Dept_HeadA |
|       2 | Dept_B    | Dept_HeadB |
|       3 | Dept_C    | Dept_HeadC |
|       4 | Dept_D    | Dept_HeadD |
+---------+-----------+------------+
```

4 rows in set (0.00 sec)

mysql> select * from Employee;

```
+--------+----------+---------+--------+
| Emp_Id | Emp_Name | Dept_Id | Salary |
+--------+----------+---------+--------+
|    101 | EmpA     |       1 |  50000 |
|    102 | EmpB     |       2 |  35000 |
|    103 | EmpC     |       3 |  70000 |
|    104 | EmpD     |       4 |  50000 |
+--------+----------+---------+--------+
```

4 rows in set (0.00 sec)

iv) Update a record

mysql> update Employee set Salary=75000.25 where Emp_Id=104;

Query OK, 1 row affected (0.01 sec)

Rows matched: 1 Changed: 1 Warnings: 0

mysql> select * from Employee where Emp_Id=104;

```
+--------+----------+---------+---------+
| Emp_Id | Emp_Name | Dept_Id | Salary  |
+--------+----------+---------+---------+
|    104 | EmpD     |       4 | 75000.2 |
+--------+----------+---------+---------+
```

1 row in set (0.00 sec)

v) Delete a record

mysql> delete from Employee where Emp_Id=103;

Query OK, 1 row affected (0.01 sec)

mysql> select *from Employee;

```
+--------+----------+---------+---------+
| Emp_Id | Emp_Name | Dept_Id | Salary  |
+--------+----------+---------+---------+
|    101 | EmpA     |       1 |   50000 |
|    102 | EmpB     |       2 |   35000 |
|    104 | EmpD     |       4 | 75000.2 |
+--------+----------+---------+---------+
```

3 rows in set (0.00 sec)

Experiment - 3

Aim :

Create a table and execute DCL statements (grant, revoke)

Program Code :

mysql> create table Employee(Emp_Id int primary key,Emp_Name varchar(50),Dept_Id int,Salary float,foreign key(Dept_Id) references Department(Dept_Id));

Query OK, 0 rows affected (0.12 sec)

mysql> insert into Employee(Emp_Id,Emp_Name,Dept_Id,Salary) values (101,'EmpA',1,50000.00),(102,'EmpB',2,35000.00),(104,'EmpD',4,75000.25);

Query OK, 3 rows affected (0.02 sec)

Records: 3 Duplicates: 0 Warnings: 0

mysql> select *from Employee;

```
+--------+----------+---------+---------+
| Emp_Id | Emp_Name | Dept_Id | Salary  |
+--------+----------+---------+---------+
|    101 | EmpA     |       1 |   50000 |
|    102 | EmpB     |       2 |   35000 |
|    104 | EmpD     |       4 | 75000.2 |
+--------+----------+---------+---------+
```

3 rows in set (0.00 sec)

mysql> create user 'xyz'@'localhost' identified by 'a1234';

Query OK, 0 rows affected (0.04 sec)

mysql> grant all privileges on Employee to 'xyz'@'localhost';

Query OK, 0 rows affected (0.02 sec)

mysql> show grants for 'xyz'@'localhost';

```
+-----------------------------------------------------------------+
| Grants for xyz@localhost                                        |
+-----------------------------------------------------------------+
| GRANT USAGE ON *.* TO `xyz`@`localhost`                         |
| GRANT ALL PRIVILEGES ON `cenna`.`Employee` TO `xyz`@`localhost` |
+-----------------------------------------------------------------+
```

2 rows in set (0.00 sec)

mysql> quit;

Bye

student@student-Veriton-M200-H310:~$ mysql -u 'xyz' -p

Enter password:

mysql> use cenna;

Reading table information for completion of table and column names

You can turn off this feature to get a quicker startup with -A

Database changed

mysql> select * from Employee;

```
+--------+----------+---------+---------+
| Emp_Id | Emp_Name | Dept_Id | Salary  |
+--------+----------+---------+---------+
|    101 | EmpA     |       1 |   50000 |
|    102 | EmpB     |       2 |   35000 |
|    104 | EmpD     |       4 | 75000.2 |
+--------+----------+---------+---------+
```

3 rows in set (0.00 sec)

mysql> quit;

Bye

sudo mysql

mysql> use cenna;

Reading table information for completion of table and column names

You can turn off this feature to get a quicker startup with -A

Database changed

mysql> revoke all privileges on Employee from 'xyz'@'localhost';

Query OK, 0 rows affected (0.02 sec)

mysql> show grants for 'xyz'@'localhost';

```
+-----------------------------------------+
| Grants for xyz@localhost                |
+-----------------------------------------+
| GRANT USAGE ON *.* TO `xyz`@`localhost` |
+-----------------------------------------+
```

1 row in set (0.00 sec)

mysql> quit;

Bye

mysql -u 'xyz' -p

Enter password:

mysql> use cenna;

ERROR 1044 (42000): Access denied for user 'xyz'@'localhost' to database cenna

mysql> quit;

Bye

Experiment - 4

Aim :

Create a table and execute TCL Statements (commit, rollback, savepoint)

Program Code :

mysql> create table Student(rollno int primary key, Name varchar(20), course varchar(20));

Query OK, 0 rows affected (0.11 sec)

mysql> insert into Student values(1,'Ammu','MCA'),(2,'Amal','MCA');

Query OK, 2 rows affected (0.02 sec)

Records: 2 Duplicates: 0 Warnings: 0

mysql> start transaction;

Query OK, 0 rows affected (0.00 sec)

mysql> savepoint a;

Query OK, 0 rows affected (0.00 sec)

mysql> select * from Student;

```
+--------+------+--------+
| rollno | Name | course |
+--------+------+--------+
|      1 | Ammu | MCA    |
|      2 | Amal | MCA    |
+--------+------+--------+
```

2 rows in set (0.00 sec)

mysql> insert into Student values(3,'Anju','MCA'),(4,'Anet','MCA');

Query OK, 2 rows affected (0.00 sec)

Records: 2 Duplicates: 0 Warnings: 0

mysql> select * from Student;

```
+--------+------+--------+
| rollno | Name | course |
+--------+------+--------+
|      1 | Ammu | MCA    |
|      2 | Amal | MCA    |
|      3 | Anju | MCA    |
|      4 | Anet | MCA    |
+--------+------+--------+
```

4 rows in set (0.00 sec)

mysql> rollback to a;

Query OK, 0 rows affected (0.01 sec)

mysql> select * from Student;

```
+--------+------+--------+
| rollno | Name | course |
+--------+------+--------+
|      1 | Ammu | MCA    |
|      2 | Amal | MCA    |
+--------+------+--------+
```

2 rows in set (0.00 sec)

mysql> commit;

Query OK, 0 rows affected (0.00 sec)

mysql> insert into Student values(5,'Arun','MCA');

Query OK, 1 row affected (0.01 sec)

mysql> select * from Student;

```
+--------+------+--------+
| rollno | Name | course |
+--------+------+--------+
|      1 | Ammu | MCA    |
|      2 | Amal | MCA    |
|      5 | Arun | MCA    |
+--------+------+--------+
```

3 rows in set (0.00 sec)

mysql> rollback;

Query OK, 0 rows affected (0.00 sec)

mysql> select * from Student;

```
+--------+------+--------+
| rollno | Name | course |
+--------+------+--------+
|      1 | Ammu | MCA    |
|      2 | Amal | MCA    |
|      5 | Arun | MCA    |
+--------+------+--------+
```

3 rows in set (0.00 sec)

Experiment - 5

Aim :

Accessing Database (SELECT, Filtering using WHERE, HAVING, GROUP BY, ORDER BY Clauses)

Program Code :

mysql> select * from Employee;

employee_id	name	gender	department	education	month_of_joining	salary_in_lakhs
1001	Ajay	M	Engineering	Doctoral	January	25
1002	Babloo	M	Engineering	UG	February	23
1003	Chhavi	F	HR	PG	March	15
1004	Dheeraj	M	HR	UG	January	15
1005	Evina	F	Marketing	UG	March	16
1006	Freddy	M	Sales	UG	December	10
1007	Garima	F	Sales	PG	March	10
1008	Hans	M	Admin	PG	November	8
1009	Ivanika	F	Admin	Intermediate	April	7
1010	Jay	M	Peon	High School	December	4

10 rows in set (0.00 sec)

1. **i) Calculate the sum of salaries of each Department**

   mysql> select department,sum(salary_in_lakhs) as salary_sum from Employee group by department;

department	salary_sum
Engineering	48
HR	30
Marketing	16
Sales	20
Admin	15
Peon	4

   6 rows in set (0.00 sec)

   ii) Find the departments in which the sum of the salaries is greater then or equal to 20 lakhs

   mysql> select department,sum(salary_in_lakhs) as salary_sum from Employee group by department having sum(salary_in_lakhs)>=20;

```
+-------------+------------+
| department  | salary_sum |
+-------------+------------+
| Engineering |         48 |
| HR          |         30 |
| Sales       |         20 |
+-------------+------------+
```

3 rows in set (0.00 sec)

iii) Display distinct department of Employee

mysql> select distinct department from Employee;

```
+-------------+
| department  |
+-------------+
| Engineering |
| HR          |
| Marketing   |
| Sales       |
| Admin       |
| Peon        |
+-------------+
```

6 rows in set (0.00 sec)

iv) Display total number of Salary of Employees

mysql> select count(salary_in_lakhs) from Employee;

```
+-------------------------+
| count(salary_in_lakhs) |
+-------------------------+
|                      10 |
+-------------------------+
```

1 row in set (0.00 sec)

v) Display the details of Employees from Employee table in which Department of the Employee is Engineering and Education is Doctoral

mysql> select * from Employee where department='Engineering' and education='Doctoral';

```
+------------+------+--------+-------------+-----------+------------------+-----------------+
| employee_id | name | gender | department  | education | month_of_joining | salary_in_lakhs |
+------------+------+--------+-------------+-----------+------------------+-----------------+
|        1001 | Ajay | M      | Engineering | Doctoral  | January          |              25 |
+------------+------+--------+-------------+-----------+------------------+-----------------+
```

1 row in set (0.00 sec)

vi) Display the details of Employees from Employee table in which Department of the Employee is Engineering or Education is Doctoral

mysql> select * from Employee where department='Engineering' or education='Doctoral';

employee_id	name	gender	department	education	month_of_joining	salary_in_lakhs
1001	Ajay	M	Engineering	Doctoral	January	25
1002	Babloo	M	Engineering	UG	February	23

2 rows in set (0.00 sec)

vii) Rename the columns "name" and "education" to "first_name" and "qualification", respectively.

mysql> select name as first_name,education as qualification from Employee;

first_name	qualification
Ajay	Doctoral
Babloo	UG
Chhavi	PG
Dheeraj	UG
Evina	UG
Freddy	UG
Garima	PG
Hans	PG
Ivanika	Intermediate
Jay	High School

10 rows in set (0.00 sec)

viii) List records of Employees whose names start with “A”.

mysql> select name from Employee where name like 'A%';

name
Ajay

1 row in set (0.01 sec)

2. Find the department in which SUM salary is greater than or equal to 20 lacs, but the education of employees is not UG.

mysql> select department,sum(salary_in_lakhs) from Employee where education<>'UG' group by department having sum(salary_in_lakhs)>=20;

department	sum(salary_in_lakhs)
Engineering	25

1 row in set (0.01 sec)

3. Find the departments in which the SUM of the salaries is greater than or equal to 15 lacs and arrange the Salary in descending order.

mysql> select department,sum(salary_in_lakhs) from Employee group by department having sum(salary_in_lakhs)>=15 order by sum(salary_in_lakhs) desc;

department	sum(salary_in_lakhs)
Engineering	48
HR	30
Sales	20
Marketing	16
Admin	15

5 rows in set (0.00 sec)

Experiment - 6

Aim :

Create a view of the tables given.

Program Code :

i) From the following table, create a view for those sales people who belong to the city of New York.

Table: salesman

salesman_id	name	city	commission
5001	James Hoog	New York	0.15
5002	Nail Knite	Paris	0.13
5003	Lauson Hen	San Jose	0.12
5005	Pit Alex	London	0.11
5006	Mc Lyon	Paris	0.14
5007	Paul Adam	Rome	0.13

mysql> create table salesman(salesman_id int primary key,name varchar(100),city varchar(100),commission decimal(2,2));

Query OK, 0 rows affected (0.10 sec)

mysql> insert into salesman values(5001,'James Hoog','New York',0.15),(5002,'Nail Knite','Paris',0.13),(5005,'Pit Alex','London',0.11),(5006,'Mc Lyon','Paris',0.14),(5007,'Paul Adam','Rome',0.13),(5003,'Lauson Hen','San Jose',0.12);

Query OK, 6 rows affected (0.02 sec)

Records: 6 Duplicates: 0 Warnings: 0

mysql> select * from salesman;

salesman_id	name	city	commission
5001	James Hoog	New York	0.15
5002	Nail Knite	Paris	0.13
5003	Lauson Hen	San Jose	0.12
5005	Pit Alex	London	0.11
5006	Mc Lyon	Paris	0.14
5007	Paul Adam	Rome	0.13

6 rows in set (0.00 sec)

mysql> create view newyorkstaff as select * from salesman where city = 'New York';

Query OK, 0 rows affected (0.03 sec)

mysql> select * from newyorkstaff;

salesman_id	name	city	commission
5001	James Hoog	New York	0.15

1 row in set (0.00 sec)

ii) From the following table, create a view that counts the number of customers in each grade.

customer_id	cust_name	city	grade	salesman_id
3001	Brad Guzan	London	100	5005
3002	Nick Rimando	New York	100	5001
3003	Jozy Altidor	Moscow	200	5007
3004	Fabian Johnson	Paris	300	5006
3005	Graham Zusi	California	200	5002
3007	Brad Davis	New York	200	5001
3008	Julian Green	London	300	5002
3009	Geoff Cameron	Berlin	100	5003

mysql> create table customer(customer_id int primary key,cust_name varchar(100),city varchar(100),grade int,salesman_id int,foreign key(salesman_id) references salesman(salesman_id));

Query OK, 0 rows affected (0.13 sec)

mysql> insert into customer values(3002,'Nick Rimando','New York',100,5001),(3007,'Brad Davis','New York',200,5001),(3005,'Graham Zuzi','California',200,5002),(3008,'Julian Green','London',300,5002),(3004,'Fabian Johnson','Paris',300,5006),(3009,'Geoff Cameron','Berlin',100,5003),(3003,'Jozy Altidor','Moscow',200,5007),(3001,'Brad Guzan','London',100,5005);

Query OK, 8 rows affected (0.01 sec)

Records: 8 Duplicates: 0 Warnings: 0

mysql> select * from customer;

customer_id	cust_name	city	grade	salesman_id
3001	Brad Guzan	London	100	5005
3002	Nick Rimando	New York	100	5001
3003	Jozy Altidor	Moscow	200	5007
3004	Fabian Johnson	Paris	300	5006
3005	Graham Zusi	California	200	5002
3007	Brad Davis	New York	200	5001
3008	Julian Green	London	300	5002
3009	Geoff Cameron	Berlin	100	5003

8 rows in set (0.00 sec)

mysql> create view grade_count(grade,number) as select grade,count(*) from customer group by grade;

Query OK, 0 rows affected (0.03 sec)

mysql> select * from grade_count;

grade	number
100	3
200	3
300	2

3 rows in set (0.01 sec)

iii) From the following table, create a view to count the number of unique customers, compute the average and the total purchase amount of customerorders by each date.

Table: orders

ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.50	2012-10-05	3005	5002
70002	65.26	2012-10-05	3002	5001
70003	2480.40	2012-10-10	3009	5003
70004	110.50	2012-08-17	3009	5003
70005	2400.60	2012-07-27	3007	5001
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005
70010	1983.43	2012-10-10	3004	5006
70011	75.29	2012-08-17	3003	5007
70012	250.45	2012-06-27	3008	5002
70013	3045.60	2012-04-25	3002	5001

mysql> create table orders(ord_no int primary key,purch_amt decimal(10,2),ord_date date,customer_id int,salesman_id int,foreign key(customer_id) references customer(customer_id),foreign key(salesman_id) references salesman(salesman_id));

Query OK, 0 rows affected (0.15 sec)

mysql> insert into orders values(70001,150.5,'2012-10-05',3005,5002),(70009,270.65,'2012-09-10',3001,5005),(70002,65.26,'2012-10-05',3002,5001),(70004,110.5,'2012-08-17',3009,5003),(70007,948.5,'2012-09-10',3005,5002),(70005,2400.6,'2012-07-27',3007,5001),(70008,5760,'2012-09-10',3002,5001),(70010,1983.43,'2012-10-

10',3004,5006),(70003,2480.4,'2012-10-10',3009,5003),(70012,250.45,'2012-06-27',3008,5002),(70011,75.29,'2012-08-17',3003,5007),(70013,3045.6,'2012-04-25',3002,5001);

mysql> select * from orders;

ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.50	2012-10-05	3005	5002
70002	65.26	2012-10-05	3002	5001
70003	2480.40	2012-10-10	3009	5003
70004	110.50	2012-08-17	3009	5003
70005	2400.60	2012-07-27	3007	5001
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005
70010	1983.43	2012-10-10	3004	5006
70011	75.29	2012-08-17	3003	5007
70012	250.45	2012-06-27	3008	5002
70013	3045.60	2012-04-25	3002	5001

12 rows in set (0.00 sec)

mysql> create view totalforday as select ord_date,count(distinct customer_id),avg(purch_amt),sum(purch_amt) from orders group by ord_date;

Query OK, 0 rows affected (0.02 sec)

mysql> select * from totalforday;

ord_date	count(distinct customer_id)	avg(purch_amt)	sum(purch_amt)
2012-04-25	1	3045.600000	3045.60
2012-06-27	1	250.450000	250.45
2012-07-27	1	2400.600000	2400.60
2012-08-17	2	92.895000	185.79
2012-09-10	3	2326.383333	6979.15
2012-10-05	2	107.880000	215.76
2012-10-10	2	2231.915000	4463.83

7 rows in set (0.00 sec)

iv) From the order table, create a view to find the salespersons who issued orders on either August 17th, 2012 or October 10th, 2012. Return salesperson ID, order number and customer ID.

Table orders :

ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.50	2012-10-05	3005	5002
70002	65.26	2012-10-05	3002	5001
70003	2480.40	2012-10-10	3009	5003
70004	110.50	2012-08-17	3009	5003
70005	2400.60	2012-07-27	3007	5001
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005
70010	1983.43	2012-10-10	3004	5006
70011	75.29	2012-08-17	3003	5007
70012	250.45	2012-06-27	3008	5002
70013	3045.60	2012-04-25	3002	5001

mysql> create view salesperson_orders as select salesman_id,ord_no,customer_id from orders where ord_date in ('2012-08-17','2012-10-10');

Query OK, 0 rows affected (0.03 sec)

mysql> select * from salesperson_orders;

salesman_id	ord_no	customer_id
5003	70003	3009
5003	70004	3009
5006	70010	3004
5007	70011	3003

4 rows in set (0.00 sec)

v) From the salesman and order tables, create a view to find the salesperson who handles a customer who makes the highest order of the day. Return order date, salesperson ID, name.

Table salesman :

salesman_id	name	city	commission
5001	James Hoog	New York	0.15
5002	Nail Knite	Paris	0.13
5003	Lauson Hen	San Jose	0.12
5005	Pit Alex	London	0.11
5006	Mc Lyon	Paris	0.14
5007	Paul Adam	Rome	0.13

Table orders :

ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.50	2012-10-05	3005	5002
70002	65.26	2012-10-05	3002	5001
70003	2480.40	2012-10-10	3009	5003
70004	110.50	2012-08-17	3009	5003
70005	2400.60	2012-07-27	3007	5001
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005
70010	1983.43	2012-10-10	3004	5006
70011	75.29	2012-08-17	3003	5007
70012	250.45	2012-06-27	3008	5002
70013	3045.60	2012-04-25	3002	5001

mysql> create view elitsalesman as select ord_date,salesman.salesman_id,name from salesman,orders ord1 where salesman.salesman_id = ord1.salesman_id and ord1.purch_amt = (select max(purch_amt) from orders ord2 where ord2.ord_date = ord1.ord_date);

Query OK, 0 rows affected (0.02 sec)

mysql> select * from elitsalesman;

ord_date	salesman_id	name
2012-07-27	5001	James Hoog
2012-09-10	5001	James Hoog
2012-04-25	5001	James Hoog
2012-10-05	5002	Nail Knite
2012-06-27	5002	Nail Knite
2012-10-10	5003	Lauson Hen
2012-08-17	5003	Lauson Hen

7 rows in set (0.00 sec)

Experiment - 7

Aim :

Subquery in MySQL

Table Employees :

emp_id	emp_name	emp_age	city	income
101	Peter	32	Newyork	200000
102	Mark	32	California	300000
103	Donald	40	Arizona	1000000
104	Obama	35	Florida	5000000
105	Linklon	32	Georgia	250000
106	Kane	45	Alaska	450000
107	Adam	35	California	5000000
108	Macculam	40	Florida	350000
109	Brayan	32	Alaska	400000
110	Stephen	40	Arizona	600000
111	Alexander	45	California	70000

Program Code :

mysql> create table Employees(emp_id int primary key, emp_name varchar(100),emp_age int,city varchar(100),income int);

Query OK, 0 rows affected (0.11 sec)

mysql> insert into Employees values(101,'Peter',32,'Newyork',200000),(102,'Mark',32,'California',300000),(103,'Donald',40,'Arizona',1000000),(104,'Obama',35,'Florida',5000000),(105,'Linklon',32,'Georgia',250000),(106,'Kane',45,'Alaska',450000),(107,'Adam',35,'California',5000000),(108,'Macculam',40,'Florida',350000),(109,'Brayan',32,'Alaska',400000),(110,'Stephen',40,'Arizona',600000),(111,'Alexander',45,'California',70000);

Query OK, 11 rows affected (0.01 sec)

Records: 11 Duplicates: 0 Warnings: 0

mysql> select * from Employees;

emp_id	emp_name	emp_age	city	income
101	Peter	32	Newyork	200000
102	Mark	32	California	300000
103	Donald	40	Arizona	1000000
104	Obama	35	Florida	5000000
105	Linklon	32	Georgia	250000
106	Kane	45	Alaska	450000
107	Adam	35	California	5000000
108	Macculam	40	Florida	350000
109	Brayan	32	Alaska	400000
110	Stephen	40	Arizona	600000
111	Alexander	45	California	70000

11 rows in set (0.00 sec)

i) Find employee detail whose id matches in a subquery.

mysql> select emp_name,emp_age,city,income from Employees where emp_id in (select emp_id from Employees);

emp_name	emp_age	city	income
Peter	32	Newyork	200000
Mark	32	California	300000
Donald	40	Arizona	1000000
Obama	35	Florida	5000000
Linklon	32	Georgia	250000
Kane	45	Alaska	450000
Adam	35	California	5000000
Macculam	40	Florida	350000
Brayan	32	Alaska	400000
Stephen	40	Arizona	600000
Alexander	45	California	70000

11 rows in set (0.00 sec)

ii) Find employee detail whose income is more than 350000 with the help of subquery.

mysql> select * from Employees where emp_id in (select emp_id from Employees where income > 350000);

emp_id	emp_name	emp_age	city	income
103	Donald	40	Arizona	1000000
104	Obama	35	Florida	5000000
106	Kane	45	Alaska	450000
107	Adam	35	California	5000000
109	Brayan	32	Alaska	400000
110	Stephen	40	Arizona	600000

6 rows in set (0.00 sec)

iii) Find employee details with maximum income using a subquery.

mysql> select * from Employees where income = (select max(income) from Employees);

emp_id	emp_name	emp_age	city	income
104	Obama	35	Florida	5000000
107	Adam	35	California	5000000

2 rows in set (0.00 sec)

Experiment - 8

Aim :

Use join queries in MySQL.

Program Code :

mysql> create table Sailors(sid int primary key,sname varchar(50),rating int,age real);

Query OK, 0 rows affected (0.10 sec)

mysql> insert into Sailors values(22,'Dustin',7,45.0);

Query OK, 1 row affected (0.02 sec)

mysql> insert into Sailors values

(29,'Brutus',1,33.0),(31,'Lubber',8,55.5),(32,'Andy',8,25.5),(58,'Rusty',10,35.0),(64,'Horatio',7,35.

0),(71,'Zorba',10,16.0),(74,'Horatio',9,35.0),(85,'Art',3,25.5),(95,'Bob',3,63.5);

Query OK, 9 rows affected (0.01 sec)

Records: 9 Duplicates: 0 Warnings: 0

mysql> select * from Sailors;

sid	sname	rating	age
22	Dustin	7	45
29	Brutus	1	33
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35
64	Horatio	7	35
71	Zorba	10	16
74	Horatio	9	35
85	Art	3	25.5
95	Bob	3	63.5

10 rows in set (0.00 sec)

mysql> create table Boats(bid int primary key,bname varchar(50),color varchar(50));

Query OK, 0 rows affected (0.10 sec)

mysql> insert into Boats

values(101,'Interlake','blue'),(102,'Interlake','red'),(103,'Clipper','green'),(104,'Marine','red');

Query OK, 4 rows affected (0.02 sec)

Records: 4 Duplicates: 0 Warnings: 0

mysql> select * from Boats;

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

4 rows in set (0.00 sec)

mysql> create table Reserves(sid int,bid int,day date,foreign key(sid) references

Sailors(sid),foreign key(bid) references Boats(bid));

Query OK, 0 rows affected (0.18 sec)

mysql> insert into Reserves values (22,101,'98-10-10');

Query OK, 1 row affected (0.02 sec)

mysql> insert into Reserves values (22,102,'98-10-10');

Query OK, 1 row affected (0.02 sec)

mysql> insert into Reserves values (22,103,'98-8-10');

Query OK, 1 row affected (0.02 sec)

mysql> insert into Reserves values (22,104,'98-7-10');

Query OK, 1 row affected (0.01 sec)

mysql> insert into Reserves values (31,102,'98-10-11');

Query OK, 1 row affected (0.02 sec)

mysql> insert into Reserves values (31,103,'98-6-11');

Query OK, 1 row affected (0.02 sec)

mysql> insert into Reserves values (31,104,'98-12-11');

Query OK, 1 row affected (0.01 sec)

mysql> insert into Reserves values (64,101,'98-5-9');

Query OK, 1 row affected (0.02 sec)

mysql> insert into Reserves values (64,102,'98-8-9');

Query OK, 1 row affected (0.02 sec)

mysql> insert into Reserves values (74,103,'98-8-9');

Query OK, 1 row affected (0.02 sec)

mysql> select * from Reserves;

sid	bid	day
22	101	1998-10-10
22	102	1998-10-10
22	103	1998-08-10
22	104	1998-07-10
31	102	1998-10-11
31	103	1998-06-11
31	104	1998-12-11
64	101	1998-05-09
64	102	1998-08-09
74	103	1998-08-09

10 rows in set (0.00 sec)

i) Count the number of distinct boat colors.

mysql> select count(distinct color) from Boats;

```
+-----------------------+
| count(distinct color) |
+-----------------------+
|                     3 |
+-----------------------+
```

1 row in set (0.01 sec)

ii) Find all information of sailors who have reserved boat number 101.

mysql> select * from Sailors where sid in (select sid from Reserves where bid = 101);

```
+-----+---------+--------+------+
| sid | sname   | rating | age  |
+-----+---------+--------+------+
|  22 | Dustin  |      7 |   45 |
|  64 | Horatio |      7 |   35 |
+-----+---------+--------+------+
```

2 rows in set (0.00 sec)

iii) Find names of sailors who have reserved at least one boat.

mysql> select sname from Sailors S,Reserves R where S.sid=R.sid;

```
+---------+
| sname   |
+---------+
| Dustin  |
| Dustin  |
| Dustin  |
| Dustin  |
| Lubber  |
| Lubber  |
| Lubber  |
| Horatio |
| Horatio |
| Horatio |
+---------+
```

10 rows in set (0.00 sec)

iv) Find names of sailors who have reserved a red boat and list in the order of their age.

mysql> select sname,age from Sailors where sid in (select sid from Reserves,Boats where Reserves.bid=Boats.bid and color='red') order by age;

sname	age
Horatio	35
Dustin	45
Lubber	55.5

3 rows in set (0.00 sec)

v) Display boat names and the names of sailors who have sailed on them.

mysql> select B.bname,S.sname from Boats B inner join Reserves R on B.bid = R.bid inner join Sailors S on S.sid = R.sid;

bname	sname
Interlake	Dustin
Interlake	Horatio
Interlake	Dustin
Interlake	Lubber
Interlake	Horatio
Clipper	Dustin
Clipper	Lubber
Clipper	Horatio
Marine	Dustin
Marine	Lubber

10 rows in set (0.00 sec)

vi) Find the ids and names of sailors who have reserved two different boats on the same day.

mysql> select distinct S.sid,S.sname from Sailors S, Reserves R1,Reserves R2 where S.sid = R1.sid and R1.day = R2.day and R1.bid <> R2.bid;

```
+-----+---------+
| sid | sname   |
+-----+---------+
|  22 | Dustin  |
|  64 | Horatio |
|  74 | Horatio |
+-----+---------+
```

3 rows in set (0.00 sec)

vii) Find the ids of sailors who have reserved a red boat or a green boat.

mysql> select R.sid from Boats B,Reserves R where R.bid = B.bid and B.color = 'red' union select R2.sid from Boats B2,Reserves R2 where R2.bid = B2.bid and B2.color = 'green';

```
+------+
| sid  |
+------+
|   22 |
|   31 |
|   64 |
|   74 |
+------+
```

4 rows in set (0.00 sec)

viii) Find the names of sailors who have reserved all boats.

mysql> select S.sname from Sailors S where not exists(select B.bid from Boats B where not exists(select R.bid from Reserves R where R.bid=B.bid and R.sid=S.sid));

```
+--------+
| sname  |
+--------+
| Dustin |
+--------+
```

1 row in set (0.00 sec)

Experiment - 9

Aim :

Write a PL/SQL procedure to display all fields from a table.

Program Code :

```
mysql> create database cenna;
Query OK, 1 row affected (0.03 sec)

mysql> use cenna;
Database changed

mysql> create table Student(rollno int primary key, Name varchar(20), course varchar(20));
Query OK, 0 rows affected (0.11 sec)

mysql> insert into Student values(1,'Ammu','MCA'),(2,'Amal','MCA');
Query OK, 2 rows affected (0.02 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> insert into Student values(5,'Arun','MCA');
Query OK, 1 row affected (0.01 sec)

mysql> delimiter //
mysql> create procedure display()
-> begin
-> select * from Student;
-> end
```

-> //

Query OK, 0 rows affected (0.03 sec)

mysql> delimiter ;

mysql> call display();

```
+--------+------+--------+
| rollno | name | course |
+--------+------+--------+
|      1 | Ammu | MCA    |
|      2 | Amal | MCA    |
|      5 | Arun | MCA    |
+--------+------+--------+
```

3 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

Experiment - 10

Aim :

Write a PL/SQL procedure to add two numbers.

Program Code :

```
mysql> delimiter //

mysql> create procedure `sum`(in a int,in b int)

-> begin

-> declare c int;

-> set c=a+b;

-> select concat('Sum of 2 numbers=',c) as result;

-> end

-> //

Query OK, 0 rows affected (0.02 sec)

mysql> delimiter ;

mysql> call `sum`(5,10);

+----------------------+
| result               |
+----------------------+
| Sum of 2 numbers=15 |
+----------------------+

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

Experiment - 11

Aim :

Write a PL/SQL procedure to check whether a number is odd or even.

Program Code :

```
mysql> delimiter //

mysql> create procedure CheckOddOrEven(in input_number int)

-> begin

-> if mod(input_number,2)=0 then

-> select 'Even' as result;

-> else

-> select 'Odd' as result;

-> end if;

-> end //

Query OK, 0 rows affected (0.04 sec)

mysql> delimiter ;

mysql> call CheckOddOrEven(2);

+--------+
| result |
+--------+
| Even   |
+--------+

1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)
```

Experiment - 12

Aim :

Write a PL/SQL procedure to find Factorial of a number.

Program Code :

```
mysql> delimiter //
mysql> create procedure Factorial(in a int)
-> begin
-> declare f int default 1;
-> while a>0 do
-> set f = f*a;
-> set a = a-1;
-> end while;
-> select concat('Factorial = ',f) as result;
-> end //
Query OK, 0 rows affected (0.02 sec)
mysql> delimiter ;
mysql> call Factorial(6);
+---------------+
| result        |
+---------------+
| Factorial=720 |
+---------------+
1 row in set (0.00 sec)
Query OK, 0 rows affected (0.00 sec)
```

Experiment - 13

Aim :

Write a PL/SQL procedure to find maximum of three values.

Program Code :

```
mysql> delimiter //
mysql> create procedure MaxOfThree(in a int,in b int,in c int)
-> begin
-> declare largest int;
-> set largest = a;
-> if b>largest then
-> set largest = b;
-> elseif c>largest then
-> set largest = c;
-> end if;
-> select largest as Largest;
-> end //
Query OK, 0 rows affected (0.02 sec)
mysql> delimiter ;
mysql> call MaxOfThree(10,15,5);
+---------+
| Largest |
+---------+
|      15 |
+---------+
1 row in set (0.00 sec)
Query OK, 0 rows affected (0.00 sec)
```

Experiment - 14

Aim :

Write a PL/SQL procedure to find the sum of digits.

Program Code :

```
mysql> delimiter //
mysql> create procedure SumOfDigits(in a int)
-> begin
-> declare digit int;
-> declare sum int default 0;
-> while a > 0 do
-> set digit = a % 10;
-> set sum = sum + digit
-> ;
-> set a = a/10;
-> end while;
-> select concat('Sum of digits = ',sum) as result;
-> end //
Query OK, 0 rows affected (0.03 sec)
mysql> delimiter ;
mysql> call SumOfDigits(121);
```

```
+-----------------+
| result          |
+-----------------+
| Sum of digits=4 |
+-----------------+
```

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

Experiment - 15

Aim :

SQL Trigger : Execution of Trigger.

The database consists of three tables: "students", "courses", and "enrollment_log". The "students" table stores information about students, including their student ID, name, and email address. The "courses" table contains details about courses offered by the university, such as course ID, title, and instructor. The "enrollment_log" table records all enrollment transactions, tracking the enrollment of students into courses. Create a trigger named "after_enrollment" that automatically inserts a log entry into the "enrollment_log" table whenever a student enrolls in a course. The log entry should include details about the enrollment action (e.g., "Student enrolled in course"), along with the timestamp of the enrollment.

Program Code :

```
mysql> create table Employees (employee_id int primary key,first_name varchar(50),last_name
varchar(50),salary decimal(10,2));
Query OK, 0 rows affected (0.09 sec)


mysql> insert into Employees(employee_id,first_name,last_name,salary) values
(1,'John','Doe',50000.00),(2,'Jane','Smith',60000.00),(3,'Michael','Johnson',55000.00);
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0


mysql> select * from Employees;
```

```
+-------------+------------+-----------+----------+
| employee_id | first_name | last_name | salary   |
+-------------+------------+-----------+----------+
|           1 | John       | Doe       | 50000.00 |
|           2 | Jane       | Smith     | 60000.00 |
|           3 | Michael    | Johnson   | 55000.00 |
+-------------+------------+-----------+----------+
```

3 rows in set (0.00 sec)

mysql> create table audit_log(log_id int primary key auto_increment,action varchar(255),

timestamp timestamp default current_timestamp);

Query OK, 0 rows affected (0.09 sec)

mysql> insert into audit_log(action,timestamp) values ('New employees

added',current_timestamp);

Query OK, 1 row affected (0.02 sec)

mysql> select * from audit_log;

```
+--------+---------------------+---------------------+
| log_id | action              | timestamp           |
+--------+---------------------+---------------------+
|      1 | New employees added | 2026-03-02 15:37:36 |
+--------+---------------------+---------------------+
```

1 row in set (0.00 sec)

mysql> delimiter //

mysql>

mysql> create trigger after_employee_insert

-> after insert on Employees

-> for each row

-> begin

-> insert into audit_log(action,timestamp)

-> values ('New employee inserted',now());

-> end //

Query OK, 0 rows affected (0.03 sec)

mysql> delimiter ;

mysql> insert into Employees (employee_id,first_name,last_name,salary)

-> values (4,'Minnu','Joseph',65000.00);

Query OK, 1 row affected (0.03 sec)

mysql> select * from Employees;

employee_id	first_name	last_name	salary
1	John	Doe	50000.00
2	Jane	Smith	60000.00
3	Michael	Johnson	55000.00
4	Minnu	Joseph	65000.00

4 rows in set (0.00 sec)

mysql> select * from audit_log;

log_id	action	timestamp
1	New employees added	2026-03-02 15:37:36
2	New employee inserted	2026-03-02 15:40:46

2 rows in set (0.00 sec)

Experiment - 16

Aim :

MySQL Trigger : BEFORE UPDATE

Program Code :

mysql> create table std1(student_id int primary key,Name varchar(20),sub1 int,sub2 int,sub3 int,sub4 int,sub5 int,total int, per_marks int,grade varchar(20));

Query OK, 0 rows affected (0.12 sec)

mysql> insert into std1 (student_id,Name,sub1,sub2,sub3,sub4,sub5) values (1,'Student1',0,0,0,0,0) ,(2,'Student2',0,0,0,0,0);

Query OK, 2 rows affected (0.01 sec)

Records: 2 Duplicates: 0 Warnings: 0

mysql> select * from std1;

student_id	Name	sub1	sub2	sub3	sub4	sub5	total	per_marks	grade
1	Student1	0	0	0	0	0	NULL	NULL	NULL
2	Student2	0	0	0	0	0	NULL	NULL	NULL

2 rows in set (0.00 sec)

mysql> delimiter //

mysql> create trigger std_before_update

-> before update on std1

-> for each row

-> begin

-> declare total_marks int;

```
-> declare per_marks int;
-> declare grade varchar(20);
-> set total_marks = new.sub1+new.sub2+new.sub3+new.sub4+new.sub5;
-> set per_marks = total_marks/5;
-> if per_marks >= 90 then
-> set grade = 'Excellent';
-> elseif per_marks >=75 and per_marks < 90 then
-> set grade = 'Very good';
-> elseif per_marks >= 60 and per_marks <= 75 then
-> set grade = 'good';
-> elseif per_marks >= 40 and per_marks <= 60 then
-> set grade = 'Average';
-> else
-> set grade = 'Not promoted';
-> end if;
-> set new.total = total_marks;
-> set new.per_marks = per_marks;
-> set new.grade = grade;
-> end;
-> //
Query OK, 0 rows affected (0.03 sec)

mysql> delimiter ;
mysql> update std1 set sub1 = 90,sub2 = 75,sub3 = 90,sub4 = 95,sub5 = 85 where student_id = 1;
Query OK, 1 row affected (0.02 sec)
```

Rows matched: 1 Changed: 1 Warnings: 0

mysql> select * from std1;

student_id	Name	sub1	sub2	sub3	sub4	sub5	total	per_marks	grade
1	Student1	90	75	90	95	85	435	87	Very good
2	Student2	0	0	0	0	0	NULL	NULL	NULL

2 rows in set (0.00 sec)

mysql> update std1 set sub1 = 80,sub2 = 95,sub3 = 89,sub4 = 92,sub5 = 90 where student_id = 2;

Query OK, 1 row affected (0.02 sec)

Rows matched: 1 Changed: 1 Warnings: 0

mysql> select * from std1;

student_id	Name	sub1	sub2	sub3	sub4	sub5	total	per_marks	grade
1	Student1	90	75	90	95	85	435	87	Very good
2	Student2	80	95	89	92	90	446	89	Very good

2 rows in set (0.00 sec)

Experiment - 17

Aim :

AFTER UPDATE Trigger

Program Code :

mysql> create table students (id int not null auto_increment,name varchar(45) not null,class int not null,email_id varchar(65) not null,primary key(id));

Query OK, 0 rows affected (0.11 sec)

mysql> insert into students (name,class,email_id) values ('Stephen',6,'stephen@gmail.com'),('Bob',7,'bob@gmail.com'),('Steven',8,'steven@gmail.com'),('Alexander',7,'alexander@gmail.com');

Query OK, 4 rows affected (0.02 sec)

Records: 4 Duplicates: 0 Warnings: 0

mysql> create table student_log(user varchar(45) not null,descriptions varchar(65) not null);

Query OK, 0 rows affected (0.09 sec)

mysql> select * from students;

```
+----+-----------+-------+---------------------+
| id | name      | class | email_id            |
+----+-----------+-------+---------------------+
|  1 | Stephen   |     6 | stephen@gmail.com   |
|  2 | Bob       |     7 | bob@gmail.com       |
|  3 | Steven    |     8 | steven@gmail.com    |
|  4 | Alexander |     7 | alexander@gmail.com |
+----+-----------+-------+---------------------+
```

4 rows in set (0.00 sec)

```
mysql> delimiter //
mysql> create trigger after_update_stdnts
-> after update
-> on students
-> for each row
-> begin
-> insert into student_log values (user(),concat('Update Student Record',old.name,'Previous
class:',old.class,'Present class',new.class));
-> end //
Query OK, 0 rows affected (0.03 sec)

mysql> delimiter ;
mysql> select * from student_log;
Empty set (0.01 sec)

mysql> update students set class = class+1;
Query OK, 4 rows affected (0.02 sec)
Rows matched: 4 Changed: 4 Warnings: 0

mysql> select * from student_log;
+----------------+-------------------------------------------------------------+
| user           | descriptions                                                |
+----------------+-------------------------------------------------------------+
| root@localhost | Update Student RecordStephenPrevious class:6Present class7  |
| root@localhost | Update Student RecordBobPrevious class:7Present class8      |
| root@localhost | Update Student RecordStevenPrevious class:8Present class9   |
| root@localhost | Update Student RecordAlexanderPrevious class:7Present class8 |
+----------------+-------------------------------------------------------------+
4 rows in set (0.00 sec)
```

mysql> select * from students;

```
+----+-----------+-------+---------------------+
| id | name      | class | email_id            |
+----+-----------+-------+---------------------+
|  1 | Stephen   |     7 | stephen@gmail.com   |
|  2 | Bob       |     8 | bob@gmail.com       |
|  3 | Steven    |     9 | steven@gmail.com    |
|  4 | Alexander |     8 | alexander@gmail.com |
+----+-----------+-------+---------------------+
```

4 rows in set (0.01 sec)

Experiment - 18

Aim :

Write a program in PL/SQL to find average salary using cursor.

Program Code :

```
mysql> delimiter //
mysql> create procedure calculate_av_salary()
-> begin
-> declare cur_salary int;
-> declare total_salary integer default 0;
-> declare num_rows integer default 0;
-> declare avg_salary integer default 0;
-> declare done boolean default false;
-> declare salary_cursor cursor for select salary from Employees;
-> declare continue handler for not found set done = true;
-> open salary_cursor;
-> fetch_loop:loop
-> fetch salary_cursor into cur_salary;
-> if done then
-> leave fetch_loop;
-> end if;
-> if cur_salary is not null then
-> set total_salary = total_salary + cur_salary;
-> set num_rows = num_rows + 1;
-> end if;
-> end loop fetch_loop;
```

```
-> close salary_cursor;

-> if num_rows > 0 then

-> set avg_salary = total_salary/num_rows;

-> end if;

-> select avg_salary;

-> end //

mysql> delimiter ;
```

```
mysql> select * from Employees;

+-------------+------------+-----------+----------+
| employee_id | first_name | last_name | salary   |
+-------------+------------+-----------+----------+
|           1 | John       | Doe       | 50000.00 |
|           2 | Jane       | Smith     | 60000.00 |
|           3 | Michael    | Johnson   | 55000.00 |
|           4 | Minnu      | Joseph    | 65000.00 |
+-------------+------------+-----------+----------+

4 rows in set (0.00 sec)
```

```
mysql> call calculate_av_salary();

+------------+
| avg_salary |
+------------+
|      57500 |
+------------+

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

Experiment - 19

Aim :

Write a program in PL/SQL to list the name of students using cursor.

Program Code :

```
mysql> delimiter $$

mysql> create procedure list_names(inout name_list varchar(4000))

-> begin

-> declare is_done integer default 0;

-> declare s_name varchar(100) default "";

-> declare stud_cursor cursor for select Name from Student;

-> declare continue handler for not found set is_done = 1;

-> open stud_cursor;

-> get_list:LOOP

-> fetch stud_cursor into s_name;

-> if is_done = 1 then

-> leave get_list;

-> end if;

-> set name_list = concat(s_name,";",name_list);

-> end loop get_list;

-> close stud_cursor;

-> end $$

Query OK, 0 rows affected (0.03 sec)


mysql> set@name_list="";

Query OK, 0 rows affected (0.00 sec)
```

```
mysql> call list_names(@name_list);

Query OK, 0 rows affected (0.00 sec)


mysql> select @name_list;

+-----------------+
| @name_list      |
+-----------------+
| Arun;Amal;Ammu; |
+-----------------+

1 row in set (0.00 sec)
```

Experiment - 20

Aim :

Create a database and collection in MongoDB.

Program Code :

i) Designing databases with NoSQL

```
> use test
switched to db test
```

ii) Make Collection with name student.

```
> db.createCollection("Student")
{ "ok" : 1 }
```

iii) Insert records into student collection.

```
> db.student.insert({rollno:101,name:"Alex",Branch:"MCA"})
WriteResult({ "nInserted" : 1 })
> db.student.insert({rollno:102,name:"Maya",Branch:"MBA"})
WriteResult({ "nInserted" : 1 })
> db.student.insert({rollno:103,name:"Niya",Branch:"BCA"})
WriteResult({ "nInserted" : 1 })
```

iv) Display the data in proper format.

```
> db.student.find().pretty()
{
"_id" : ObjectId("681d9a85be9d4402d973e957"),
"rollno" : 101,
```

"name" : "Alex",

"Branch" : "MCA"

}

{

"_id" : ObjectId("681d9aa8be9d4402d973e958"),

"rollno" : 102,

"name" : "Maya",

"Branch" : "MBA"

}

{

"_id" : ObjectId("681d9ac2be9d4402d973e959"),

"rollno" : 103,

"name" : "Niya",

"Branch" : "BCA"

}

Experiment - 21

Aim :

Designing NoSQL database – Employee Management.

Program Code :

i) Create a NoSQL database named "Employee".

```
>use Employee
switched to db Employee
```

ii) Create a collection named "EMPL" with fields: "Empno", "Name", "Salary", and "Role".

```
>db.createCollection("EMPL")
{ "ok" : 1 }
```

iii) Insert 3 records into the "EMPL" collection.

```
> db.EMPL.insertMany([{"Empno":1,"Name":"John
Doe","Salary":60000,"Role":"Manager"},{"Empno":2,"Name":"Alice
Smith","Salary":50000,"Role":"Developer"},{"Empno":3,"Name":"Bob","Salary":70000,"R
ole":"System Analyst"}])
{
"acknowledged" : true,
"insertedIds" : [
ObjectId("67f3617eddab1f16cd74988f"),
ObjectId("67f3617eddab1f16cd749890"),
ObjectId("67f3617eddab1f16cd749891")
]}
```

iv) Display the data from the "EMPL" collection in a proper format.

```
> db.EMPL.find().pretty()
{
"_id" : ObjectId("67f3617eddab1f16cd74988f"),
"Empno" : 1,
"Name" : "John Doe",
"Salary" : 60000,
"Role" : "Manager"
}
{
"_id" : ObjectId("67f3617eddab1f16cd749890"),
"Empno" : 2,
"Name" : "Alice Smith",
"Salary" : 50000,
"Role" : "Developer"
}
{
"_id" : ObjectId("67f3617eddab1f16cd749891"),
"Empno" : 3,
"Name" : "Bob",
"Salary" : 70000,
"Role" : "System Analyst"
}
```

v) Retrieve employees from the "EMPL" collection based on their roles.

```
> db.EMPL.find({"Role":"Manager"}).pretty()
{
"_id" : ObjectId("67f3617eddab1f16cd74988f"),
"Empno" : 1,
"Name" : "John Doe",
"Salary" : 60000,
"Role" : "Manager"
}
```

vi) Update the salary of an employee in the "EMPL" collection.

```
> db.EMPL.updateOne({"Name":"Alice Smith"},{$set:{"Salary":55000}})
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
```

Experiment - 22

Aim :

Perform CRUD Operations – Product Catalog.

Program Code :

i) Create a NoSQL database named "ProductCatalog".

```
> use productCatalog
switched to db productCatalog
```

ii) Create a collection named "Products" with fields: "ProductID", "ProductName", "Price", and "Quantity".

```
> db.createCollection("Products")
{ "ok" : 1 }
```

iii) Insert several records into the "Products" collection.

```
>db.Products.insertMany([{"ProductID":1,"ProductName":"Laptop","Price":1000,"Quantity":20
},{"ProductID":2,"ProductName":"Smartphone","price":500,"Quantity":30},{"ProductID":3,"Pr
oductName":"washing machine","price":50000,"Quantity":50}])
{
"acknowledged" : true,
"insertedIds" : [
ObjectId("67f367ac4b7b5a11715f4408"),
ObjectId("67f367ac4b7b5a11715f4409"),
ObjectId("67f367ac4b7b5a11715f440a")
]
}
```

iv) Display the data from the "Products" collection.

```
> db.Products.find().pretty()
{
"_id" : ObjectId("67f367ac4b7b5a11715f4408"),
"ProductID" : 1,
"ProductName" : "Laptop",
"Price" : 1000,
"Quantity" : 20
}
{
"_id" : ObjectId("67f367ac4b7b5a11715f4409"),
"ProductID" : 2,
"ProductName" : "Smartphone",
"price" : 500,
"Quantity" : 30
}
{
"_id" : ObjectId("67f367ac4b7b5a11715f440a"),
"ProductID" : 3,
"ProductName" : "washing machine",
"price" : 50000,
"Quantity" : 50
}
```

v) Update the details of a specific product. (For example, increase the quantity of laptops by 10)

```
> db.Products.updateOne({"ProductName":"Laptop"},{$inc:{"Quantity":10}})
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
> db.Products.find({"ProductID":1}).pretty()
{
"_id" : ObjectId("67f367ac4b7b5a11715f4408"),
"ProductID" : 1,
"ProductName" : "Laptop",
"Price" : 1000,
"Quantity" : 30
}
```

vi) Delete a product from the collection. (For example, remove the smartphone from the catalog.)

```
> db.Products.deleteOne({"ProductName":"Smartphone"})
{ "acknowledged" : true, "deletedCount" : 1 }
> db.Products.find({"ProductID":2}).pretty()
>
```

Experiment - 23

Aim :

Usage of Aggregate functions.

Program Code :

```
> use Students
switched to db Students
> db.Students.insertMany([{"Name":"Tony","age":17,"id":1,"sec":"A","subject":["physics","maths
"]},{"Name":"Alice","age":18,"id":2,"sec":"A","subject":["physics","maths"]},{"Name":"John",
"age":17,"id":3,"sec":"B","subject":["chemistry","maths"]},{"Name":"smith","age":17,"id":4,"se
c":"B","subject":["chemistry","maths"]},{"Name":"Bob","age":18,"id":5,"sec":"C","subject":["c
omputer","maths"]},{"Name":"Doe","age":18,"id":6,"sec":"C","subject":["computer","maths"]}]
)
{
"acknowledged" : true,
"insertedIds" : [
ObjectId("67f5fd8a8019c484b4a2fc04"),
ObjectId("67f5fd8a8019c484b4a2fc05"),
ObjectId("67f5fd8a8019c484b4a2fc06"),
ObjectId("67f5fd8a8019c484b4a2fc07"),
ObjectId("67f5fd8a8019c484b4a2fc08"),
ObjectId("67f5fd8a8019c484b4a2fc09")
]
}
```

i) Displaying the total number of students in one section only

```
> db.Students.aggregate([{$match:{sec:"B"}},{$count:"Total Student in sec:B"}])
{ "Total Student in sec:B" : 2 }
```

ii) Displaying the total number of students in both the sections and maximum age from both section

```
> db.Students.aggregate([{$group:{_id:"$sec",total_st: {$sum:1},max_age:{$max:"$age"}}}])
{ "_id" : "B", "total_st" : 2, "max_age" : 17 }
{ "_id" : "C", "total_st" : 2, "max_age" : 18 }
{ "_id" : "A", "total_st" : 2, "max_age" : 18 }
```

iii) Displaying details of students whose age is greater than 30 using match stage

```
> db.Students.aggregate([{$match:{age:{$gt:17}}}]).pretty()
{
"_id" : ObjectId("67f5fd8a8019c484b4a2fc05"),
"Name" : "Alice",
"age" : 18,
"id" : 2,
"sec" : "A",
"subject" : [
"physics",
"maths"
]
}
{
```

"_id" : ObjectId("67f5fd8a8019c484b4a2fc08"),

"Name" : "Bob",

"age" : 18,

"id" : 5,

"sec" : "C",

"subject" : [

"computer",

"maths"

]

}

{

"_id" : ObjectId("67f5fd8a8019c484b4a2fc09"),

"Name" : "Doe",

"age" : 18,

"id" : 6,

"sec" : "C",

"subject" : [

"computer",

"maths"

]

}

iv) Sorting the students on the basis of age

```
> db.Students.aggregate([{'$sort':{'age':1}}])
{ "_id" : ObjectId("67f5fd8a8019c484b4a2fc04"), "Name" : "Tony", "age" : 17, "id" : 1, "sec"
: "A", "subject" : [ "physics", "maths" ] }
{ "_id" : ObjectId("67f5fd8a8019c484b4a2fc06"), "Name" : "John", "age" : 17, "id" : 3, "sec"
: "B", "subject" : [ "chemistry", "maths" ] }
{ "_id" : ObjectId("67f5fd8a8019c484b4a2fc07"), "Name" : "smith", "age" : 17, "id" : 4, "sec"
: "B", "subject" : [ "chemistry", "maths" ] }
{ "_id" : ObjectId("67f5fd8a8019c484b4a2fc05"), "Name" : "Alice", "age" : 18, "id" : 2, "sec"
: "A", "subject" : [ "physics", "maths" ] }
{ "_id" : ObjectId("67f5fd8a8019c484b4a2fc08"), "Name" : "Bob", "age" : 18, "id" : 5, "sec" :
"C", "subject" : [ "computer", "maths" ] }
{ "_id" : ObjectId("67f5fd8a8019c484b4a2fc09"), "Name" : "Doe", "age" : 18, "id" : 6, "sec" :
"C", "subject" : [ "computer", "maths" ] }
```

v) Counting the total numbers of documents.

```
> db.Students.count()
6
```

vi) Displaying distinct names and ages (non-repeating).

```
> db.Students.distinct("Name")
[ "Alice", "Bob", "Doe", "John", "Tony", "smith" ]
> db.Students.distinct("age")
[ 17, 18 ]
```

Experiment - 24

Aim :

Create users and roles in the Employee database.

Program Code :

```
> use Employee
switched to db Employee
> db.createCollection("EMPL")
{ "ok" : 1 }
> db.EMPL.insertMany([{"Empno":1,"Name":"John
Doe","Salary":60000,"Role":"Manager"},{"Empno":1,"Name":"John
Doe","Salary":60000,"Role":"Manager"}])
{
"acknowledged" : true,
"insertedIds" : [
ObjectId("67f60b088019c484b4a2fc0a"),
ObjectId("67f60b088019c484b4a2fc0b")
]
}
> db.EMPL.find().pretty()
{
"_id" : ObjectId("67f60b088019c484b4a2fc0a"),
"Empno" : 1,
"Name" : "John Doe",
"Salary" : 60000,
"Role" : "Manager"
```

```
}
{
"_id" : ObjectId("67f60b088019c484b4a2fc0b"),
"Empno" : 1,
"Name" : "John Doe",
"Salary" : 60000,
"Role" : "Manager"
}
> db.createUser({user:"admin"
... ,pwd:"admin123",roles:["readWrite","dbAdmin"]});
Successfully added user: { "user" : "admin", "roles" : [ "readWrite", "dbAdmin" ] }
> db.createUser({user:"developer",pwd:"dev123",roles:["readWrite"]});
Successfully added user: { "user" : "developer", "roles" : [ "readWrite" ] }
> show users
{
"_id" : "Employee.admin",
"userId" : UUID("55d13193-d7a4-4358-bdf0-153e635f1009"),
"user" : "admin",
"db" : "Employee",
"roles" : [
{
"role" : "readWrite",
"db" : "Employee"
},
{
```

"role" : "dbAdmin",

"db" : "Employee"

}

],

"mechanisms" : [

"SCRAM-SHA-1",

"SCRAM-SHA-256"

]

}

{

"_id" : "Employee.developer",

"userId" : UUID("5a7fd4ed-6fe5-4cec-98c0-db34a92b935a"),

"user" : "developer",

"db" : "Employee",

"roles" : [

{

"role" : "readWrite",

"db" : "Employee"

}

],

"mechanisms" : [

"SCRAM-SHA-1",

"SCRAM-SHA-256"

]

}

Experiment - 25

Aim :

To build a sample collections/documents to perform the shell commands like replica set, indexing.

Program Code :

```
db.createCollection("students")
{ ok: 1 }
db.students.insertMany([
{ _id:1, name:"Kiran", dept:"CSE", age:21, city:"Kochi" },
{ _id:2, name:"Anitha", dept:"ECE", age:22, city:"TVM" },
{ _id:3, name:"Rahul", dept:"CSE", age:23, city:"Kochi" },
{ _id:4, name:"Sneha", dept:"ME", age:21, city:"Calicut" }
])
{
acknowledged: true,
insertedIds: {
'0': 1,
'1': 2,
'2': 3,
'3': 4
}
}
db.createCollection("courses")
{ ok: 1 }
db.courses.insertMany([
{ course_id:"CS101", course_name:"DBMS", credits:4 },
```

```
{ course_id:"CS102", course_name:"OS", credits:3 },
{ course_id:"EC201", course_name:"Signals", credits:4 }
])
{
acknowledged: true,
insertedIds: {
'0': ObjectId('69d66a4148b1a0e32f8ce5af'),
'1': ObjectId('69d66a4148b1a0e32f8ce5b0'),
'2': ObjectId('69d66a4148b1a0e32f8ce5b1')
}
}
db.students.createIndex({ name:1 })
name_1
db.students.createIndex({ dept:1, age:-1 })
dept_1_age_-1
db.students.getIndexes()
[
{ v: 2, key: { _id: 1 }, name: '_id_' },
{ v: 2, key: { name: 1 }, name: 'name_1' },
{ v: 2, key: { dept: 1, age: -1 }, name: 'dept_1_age_-1' }
]
db.students.find({ name:"Kiran" })
[ { _id: 1, name: 'Kiran', dept: 'CSE', age: 21, city: 'Kochi' } ]
mongod --port 27017 --dbpath /data/db1 --replSet rs0
{"msg":"MongoDB starting","port":27017}
```

```
{"msg":"Waiting for connections","port":27017}
mongod --port 27018 --dbpath /data/db2 --replSet rs0
{"msg":"MongoDB starting","port":27018}
{"msg":"Waiting for connections","port":27018}
mongod --port 27019 --dbpath /data/db3 --replSet rs0
{"msg":"MongoDB starting","port":27019}
{"msg":"Waiting for connections","port":27019}
mongosh --port 27017
Current Mongosh Log ID: 1234567890
Connecting to: mongodb://127.0.0.1:27017/
Using MongoDB: 6.x.x
Using Mongosh: 2.x.x
rs.initiate()
{ ok: 1 }
rs.add("localhost:27018")
{ ok: 1 }
rs.add("localhost:27019")
{ ok: 1 }
rs.status()
{
set: "rs0",
date: ISODate("2026-04-08T10:20:00Z"),
myState: 1,
members: [
{
```

```
_id: 0,
name: "localhost:27017",
stateStr: "PRIMARY"
},
{
_id: 1,
name: "localhost:27018",
stateStr: "SECONDARY"
},
{
_id: 2,
name: "localhost:27019",
stateStr: "SECONDARY"
}
],
ok: 1
}
db.students.insertOne({name:"Arun", dept:"CE", age:22})
{
acknowledged: true,
insertedId: ObjectId("64f8c2b1a12bc34d5678abcd")
}
mongosh --port 27019
{ ok: 1 }
db.getMongo().setReadPref("secondary")
```

```
use test
switched to db test
db.students.find()
[
{ _id: 1, name: "Kiran", dept: "CSE", age: 21, city: "Kochi" },
{ _id: 2, name: "Anitha", dept: "ECE", age: 22, city: "TVM" },
{ _id: 3, name: "Rahul", dept: "CSE", age: 23, city: "Kochi" },
{ _id: 4, name: "Sneha", dept: "ME", age: 21, city: "Calicut" },
{
_id: ObjectId("64f8c2b1a12bc34d5678abcd"),
name: "Arun",
dept: "CE",
age: 22
}
]
```

Experiment - 26

Aim :

Develop a simple PHP application using MongoDB (NoSQL) to insert and retrieve student records.

Program Code :

```
<?php
$manager = new MongoDB\Driver\Manager("mongodb://localhost:27017");
if(isset($_POST['submit'])){
   $bulk = new MongoDB\Driver\BulkWrite;
   $doc = [
      "name" => $_POST['name'],
      "roll" => $_POST['roll'],
      "dept" => $_POST['dept']
   ];
   $bulk->insert($doc);
   $manager->executeBulkWrite("college.students", $bulk);
   echo "Record Inserted Successfully<br>";
}
$query = new MongoDB\Driver\Query([]);
$rows = $manager->executeQuery("college.students", $query);
?>
<html>
<body>
<h2>Student Form</h2>
<form method="post">
Name: <input type="text" name="name"><br>
Roll: <input type="text" name="roll"><br>
Dept: <input type="text" name="dept"><br>
<input type="submit" name="submit">
</form>
<h2>Stored Records</h2>
<?php
foreach ($rows as $row) {
   echo $row->name . " - " . $row->roll . " - " . $row->dept . "<br>";
```

```
}
?>
</body>
</html>
```

Output :

Record Inserted Successfully

Student Form

Name:
Roll:
Dept:
Submit Query

Stored Records

cenna - 17 - mca